Programação Orientada a Objetos

Introdução

O que vamos aprender

- Conceitos de POO;
  - Classes
  - Objetos
  - Métodos
  - Atributos
- Motivação para utilizar POO;
  - Que problemas esse paradigma resolve
- Atributos e métodos

O que vamos aprender

- Construção e organização
  - Método construtor
  - Boas práticas
- Métodos especiais e Classes estáticas
- Encapsulamento
- Herança
- Polimorfismo
- Abstração

Ferramentas

- simplicidade na instalação e execução
- ferramentas leves
- acompanhamento com outros professores

python

Para essa aula

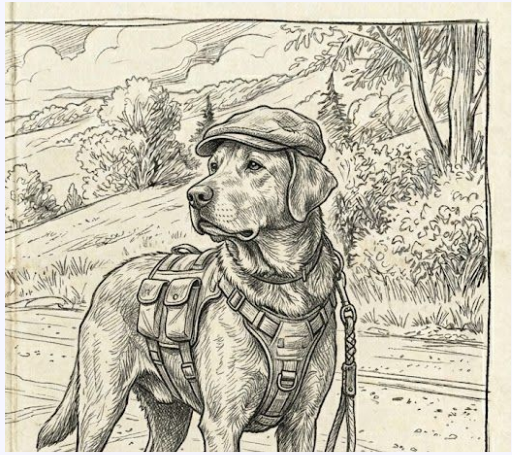
- O que é Orientação a Objetos
- Diferença entre o paradigma procedural e OO
- Porque usar OO
- Relação com elementos do mundo real

O que é Orientação a objetos?

O que essas 3 coisas do dia a dia têm em comum?

Carro

Cachorro

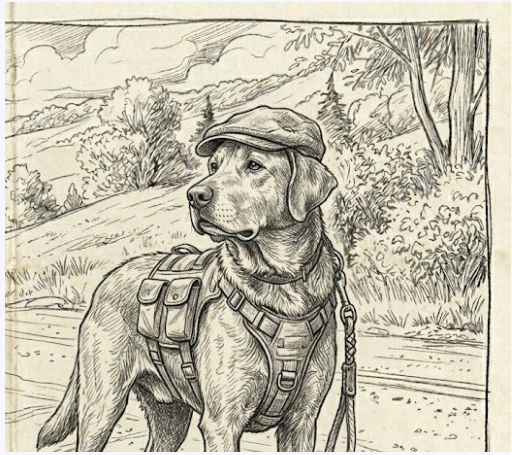


Pessoa

O que essas 3 coisas do dia a dia têm em comum?

Carro

Cachorro



Pessoa

- **Possuem características**
- **Realizam ações**

Um objeto é algo que busca representar uma entidade do mundo real dentro do sistema

Um sistema orientado a objetos nada mais é do que um conjunto de objetos interagindo entre si

Dúvidas?

Procedural
X
Orientação a objetos

```python
# Função para consultar saldo
def consultar_saldo(numero):
    if numero not in contas:
        print("Conta não encontrada.")
        return

    saldo = contas[numero]["saldo"]
    print(f"Saldo da conta {numero}: R${saldo}")


# Função para transferir
def transferir(origem, destino, valor):
    if origem not in contas or destino not in contas:
        print("Conta de origem ou destino não encontrada.")
        return

    if contas[origem]["saldo"] < valor:
        print("Saldo insuficiente para transferência.")
        return

    contas[origem]["saldo"] -= valor
    contas[destino]["saldo"] += valor

    print(f"Transferência de R${valor} realizada de {origem} para {destino}.")


# ========================
# Execução do programa
# ========================


#chamando função por função
criar_conta(1, "João", 1000)
criar_conta(2, "Maria", 500)

depositar(1, 200)
sacar(2, 100)

consultar_saldo(1)
consultar_saldo(2)

transferir(1, 2, 300)

consultar_saldo(1)
consultar_saldo(2)
```

Não existem classes nem objetos

Lógica organizada em funções independentes

Fluxo do programa é sequencial

```python
class Conta:
    def __init__(self, numero, titular, saldo_inicial=0):
        self.numero = numero
        self.titular = titular
        self.saldo = saldo_inicial

    def depositar(self, valor):
        self.saldo += valor
        print(f"Depósito de R${valor} realizado. Novo saldo: R${self.saldo}")

    def sacar(self, valor):
        if self.saldo < valor:
            print("Saldo insuficiente.")
            return

        self.saldo -= valor
        print(f"Saque de R${valor} realizado. Novo saldo: R${self.saldo}")

    def consultar_saldo(self):
        print(f"Saldo da conta {self.numero}: R${self.saldo}")


class Banco:
    def __init__(self):
        self.contas = {}
```

```python
banco = Banco()

banco.criar_conta(1, "João", 1000)
banco.criar_conta(2, "Maria", 500)

banco.depositar(1, 200)
banco.sacar(2, 100)

banco.consultar_saldo(1)
banco.consultar_saldo(2)

banco.transferir(1, 2, 300)

banco.consultar_saldo(1)
banco.consultar_saldo(2)
```

Criamos classes

Dados são **encapsulados** dentro dos objetos

Separação de responsabilidades

Antes: Dados perdidos
Agora: Dados encapsulados

Eu sei a quem aquela informação pertence

Na programação procedural, funções operam sobre dados. Na orientação a objetos, os objetos possuem seus próprios dados e comportamentos

```c
struct Data {
  //o tamanho da struct é a soma das propriedades dela
  //3 inteiros = 12 bytes
  short dia;
  short mes;
  short ano;
  short isValid;

  // essa struct precisa de funcoes especificas pra ela, como tratamentos que limitam o mes, o dia, etc
} Data;

void initData(struct Data* dt) {
    dt->isValid = 0; // false
}

void change(struct Data* dt, int dia, int mes, int ano) {
    dt->ano = ano;
    dt->mes = mes;
    dt->dia = dia;

    dt->isValid = 0;
    if ((dt->dia >= 1 && dt->dia <= 31) && (dt->mes >= 1 && dt->mes <= 12)) {
        dt->isValid = 1;
    }
}

void printData(const struct Data* dt) {
    if (dt->isValid) {                                  ⚠ Branch condition evaluates to a garbage value
        printf("%02d/%02d/%d\n", dt->dia, dt->mes, dt->ano);
    } else {
        printf ("Data invalida\n");
    }
}

int main()
{

    struct Data dt;
    // initData(&dt);
    // change(&dt, 21, 3, 2001);

    printf("size: %llu\n", sizeof(dt));

```

```cpp
struct Data {
    // o tamanho da struct é a soma das propriedades dela
    // 3 inteiros = 12 bytes

    void init() { this->isValid = false; }

    void change(int dia, int mes, int ano) {
        this->ano = ano;
        this->mes = mes;
        this->dia = dia;

        isValid = false;
        if ((this->dia >= 1 && this->dia <= 31) && (this->mes >= 1 && this->mes <= 12)) {
            isValid = true;
        }
    }

    void print() const {
        if (isValid) {
            printf("%02d/%02d/%d\n", dia, mes, ano);
        } else {
            printf("Data invalida\n");
        }
    }

  private:
    bool isValid;
    short dia;
    short mes;
    short ano;
};

int main() {

    struct Data dt;
    dt.change(21, 3, 2026);

    printf("size: %llu\n", sizeof(dt));
    dt.print();
}
```

Dúvidas?

Por que usar POO?

Sistema que calcula média de notas do alunos

```python
def calcular_media(notas):
    return sum(notas) / len(notas)


def verificar_aprovacao(media):
    if media >= 7:
        return "Aprovado"
    else:
        return "Reprovado"


def executar():
    print("=== Cálculo de Média ===")

    notas = []

    for i in range(3):
        nota = float(input(f"Digite a nota {i+1}: "))
        notas.append(nota)

    media = calcular_media(notas)
    status = verificar_aprovacao(media)

    print(f"\nMédia: {media:.2f}")
    print(f"Situação: {status}")


executar()
```

Sistema que calcula média de notas do alunos

```python
def calcular_media(notas):
    return sum(notas) / len(notas)


def verificar_aprovacao(media):
    if media >= 7:
        return "Aprovado"
    else:
        return "Reprovado"


def executar():
    print("=== Cálculo de Média ===")

    notas = []

    for i in range(3):
        nota = float(input(f"Digite a nota {i+1}: "))
        notas.append(nota)

    media = calcular_media(notas)
    status = verificar_aprovacao(media)

    print(f"\nMédia: {media:.2f}")
    print(f"Situação: {status}")


executar()
```

Classe Aluno
Classe Calculadora

- mais código
- mais complexidade
- nenhum benefícios

É simples demais, a orientação a objetos não se paga

Usar X Não usar

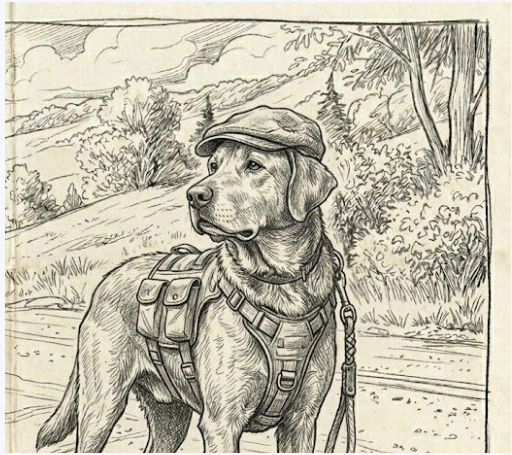
Situação	Usar POO	Não usar POO
Complexidade do sistema	Médio/Grande	scripts simples
Entidades de domínio	Existem objetos claros	Não há entidades claras
Manutenção futura	Código vai evoluir	Código descartável
Reutilização	Reutilizar partes do código	Uso único
Estado dos dados	Precisa manter estados complexos	Pouco ou nenhum estado
Organização	Precisa separar as responsabilidades	Código curto, já é legível
Escalabilidade	Sistema pode aumentar	Não vai crescer
Regras de negócio	Muitas regras e validações	pouca lógica, só processamento
Modelagem do mundo real	Representa coisas reais	Não representa entidades

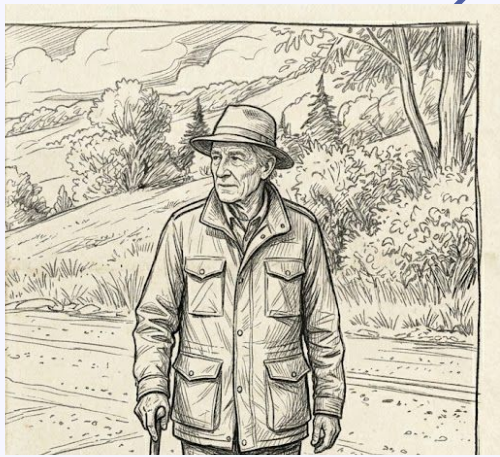
Modelar o mundo real dentro do código

O código começa a fazer sentido a partir do momento que olhamos para o problema, por conta da relação com o mundo real

Dúvidas?

Relação com os elementos do mundo real





Características?
Ações?